

Got it — you have an **EF Core** entity model where the navigation property exists, but **no explicit foreign key property** is defined in the class. You want to create a new entity and associate it with an existing one **by ID only**, without loading the full related entity from the database.

Problem

In EF Core, if your model looks like this:

```
public class Order
{
    public int Id { get; set; }

    // No CustomerId property here
    public Customer Customer { get; set; }
}

public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

The database has a `CustomerId` column (created by EF Core from the relationship), but your C# model doesn't expose it.

Solution 1 — Attach a Stub Entity

You can avoid fetching the related entity from the database by creating a **stub** with only the ID set and attaching it to the context.

```
using (var context = new AppDbContext())
{
    var order = new Order
    {
        // Assign a stub Customer with only the Id
        Customer = new Customer { Id = 5 }
    };

    // Mark the stub as unchanged so EF doesn't try to insert it
    context.Attach(order.Customer);

    context.Orders.Add(order);
    context.SaveChanges();
}
```

Why it works:

EF Core sees that Customer is already tracked as Unchanged and will only set the foreign key in the Order insert.

Solution 2 — Add an Explicit Foreign Key Property

If you can modify the model, the cleaner approach is to explicitly add the FK property:

```
public class Order
{
    public int Id { get; set; }

    public int CustomerId { get; set; } // Explicit FK
}
```

```
public Customer Customer { get; set; }
}
```

Then you can simply do:

```
var order = new Order
{
    CustomerId = 5
};

context.Orders.Add(order);
context.SaveChanges();
```

This avoids creating a stub entity and is more efficient.

Solution 3 — Use Entry API to Set FK Without Property

If you cannot change the model and don't want to create a stub object:

```
var order = new Order();

context.Entry(order)
    .Reference(o => o.Customer)
    .TargetEntry
    .Property("Id") // Shadow property name
    .CurrentValue = 5;
```

```
context.Orders.Add(order);  
context.SaveChanges();
```

Here "Id" is the PK of Customer. EF Core internally creates a **shadow foreign key property** (likely named "CustomerId"). You can set it via `Property("CustomerId")` if you know the name.

✅ **Recommendation:**

If possible, **add the explicit FK property** — it's the most maintainable and avoids EF Core's shadow property complexity.

If you want, I can give you a **full runnable EF Core example** showing all three approaches so you can test them.

Do you want me to prepare that?